

Practical - 4

Practical Title: JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Problem Definition: Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript.

A. Theory

1.How are DOM elements selected and modified?

Answer: DOM elements are selected using methods such as `getElementById()`, `getElementsByClassName()`, `getElementsTagName()`, `querySelector()`, and `querySelectorAll()`. After selecting an element, JavaScript can modify its content using properties such as `innerHTML` and `textContent`, change its attributes, or modify its CSS styles.

Example:

```
const element = document.getElementById("demo");  
element.innerHTML = "Hello World!";
```

Here, `getElementById()` selects the element and `innerHTML` changes its content.

Reference Link:

https://www.w3schools.com/js/js_htmlDOM_elements.asp

https://www.w3schools.com/js/js_htmlDOM_methods.asp

2. Are event listeners attached correctly?

Answer: Yes, event listeners are correctly attached when `addEventListener()` is used with the event name and the function that should execute when the event occurs.

Example:

```
const button = document.getElementById("myBtn");
```

```
button.addEventListener("click", function() {  
    alert("Button clicked!");  
});
```

According to W3Schools, `addEventListener()` attaches an event handler to an element. It also allows multiple event handlers to be attached without overwriting existing handlers. The event name should be written as "click", not "onclick"

Reference Link:

https://www.w3schools.com/js/js_html_dom_eventlistener.asp

3. Is `localStorage` used for remembering the theme choice?

Answer: `localStorage` can be used to remember the user's light/dark theme choice even after the browser is closed and reopened. W3Schools states that `localStorage` stores data with no expiration date.

Example:

```
// Save theme
```

```
localStorage.setItem("theme", "dark");
```

```
// Get saved theme
```

```
let theme = localStorage.getItem("theme");
```

The `setItem()` method stores the value, while `getItem()` retrieves it.

For a theme switcher, you can therefore save "dark" or "light" in `localStorage` and apply the saved choice when the page loads.

Reference Link:

https://www.w3schools.com/js/js_api_web_storage.asp

4. Does interactivity improve usability without breaking accessibility?

Answer: Yes, if the interactive elements are implemented accessibly. JavaScript interactivity such as menus, modals, sliders, and theme switches can make a website easier to use. However, interactive elements should still use appropriate semantic HTML elements and remain usable with keyboard navigation and screen readers.

For example, W3Schools recommends using a real `<button>` for a button rather than using a `<div>`. A `<button>` is focusable, clickable, recognized as a button by screen readers, and can be operated using the keyboard.

Therefore, JavaScript should enhance the user experience rather than make the website dependent only on mouse clicks or visual effects.

Reference Link:

https://www.w3schools.com/html/html_accessibility.asp

B. Implementation/ Output/Result

```

C.
D. let loginform=document.getElementById("loginform");
E.
F. if (loginform)
G. {
H.     loginform.addEventListener("submit", function(event)
I.     {
J.         let username=document.getElementById("username");
K.         let password=document.getElementById("password");
L.         let message=document.getElementById("message");
M.         event.preventDefault();
N.
O.         if(username.value=="")
P.         {
Q.             message.innerHTML= "Please enter the username";
R.             message.style.color="Blue";
S.             return;
T.         }
U.
V.         if(password.value=="")
W.         {
X.             message.innerHTML="Please enter the Password";
Y.             return;
Z.         }
AA.         message.innerHTML="Login Succesfully!";
BB.     });
CC.
DD. }
EE.
FF. let signupform=document.getElementById("signupform");
GG.
HH. if(signupform)
II. {
JJ.     let email=document.getElementById("email");
KK. let crpassword=document.getElementById("createpassword");
LL. let copassword=document.getElementById("confirmpassword");
MM.
NN.     signupform.addEventListener("submit", function(event)
OO.     {

```

```

PP.      event.preventDefault();
QQ.
RR.      if(email.value=="")
SS.      {
TT.          message.innerHTML= "Please enter the valid email ID";
UU.          message.style.color="Blue";
VV.          return;
WW.      }
XX.
YY.      if(crpassword.value=="")
ZZ.      {
AAA.          message.innerHTML="Please enter the Password";
BBB.          return;
CCC.      }
DDD.
EEE.      if(copassword.value=="")
FFF.      {
GGG.          message.innerHTML="Please confirm the Password";
HHH.          return;
III.      }
JJJ.
KKK.      if(crpassword.value!=copassword.value)
LLL.      {
MMM.          message.innerHTML="Passwords do not match";
NNN.          return;
OOO.      }
PPP.      message.innerHTML="Account Created Succesfully!";
QQQ.  });
RRR.  }
SSS.
TTT.  let wdf=document.getElementById("wdf");
UUU.  let cpp=document.getElementById("cpp");
VVV.  let dsa=document.getElementById("dsa");
WWW.
XXX.  function checkAttendance(attendance,statusId)
YYY.  {
ZZZ.      let percentage=parseInt(attendance.innerHTML)
AAAA.
BBBB.      if(percentage>=75)
CCCC.      {
DDDD.          document.getElementById(statusId).innerHTML="Good
              Attendance";
EEEE.          document.getElementById(statusId).style.color="green";
FFFF.      }

```

```

GGGG.     else
HHHH.     {
IIII.         document.getElementById(statusId).innerHTML="You need to
attend the classes regularly";
JJJJ.         document.getElementById(statusId).style.color="red";
KKKK.     }
LLLL. }
MMMM.

NNNN. function calculateGrade(percentage,gradeId)
OOOO. {
PPPP.     let marks=parseInt(percentage.innerHTML)
QQQQ.
RRRR.     if(marks>=90)
SSSS.     {
TTTT.         document.getElementById(gradeId).innerHTML="A+";
UUUU.         document.getElementById(gradeId).style.color="green";
VVVV.     }
WWWW.     else if(marks>=80)
XXXX.     {
YYYY.         document.getElementById(gradeId).innerHTML="A";
ZZZZ.         document.getElementById(gradeId).style.color="green";
AAAAA.     }
BBBBB.     else if(marks>=70)
CCCCC.     {
DDDDD.         document.getElementById(gradeId).innerHTML="B";
EEEE.         document.getElementById(gradeId).style.color="green";
FFFF.     }
GGGGG.     else
HHHHH.     {
IIIII.         document.getElementById(gradeId).innerHTML="C";
JJJJJ.         document.getElementById(gradeId).style.color="green";
KKKKK.     }
LLLLL.
MMMMM. }
NNNNN.
OOOOO. if(document.getElementById("wdfStatus"))
PPPPP. {
QQQQQ.
RRRRR.     checkAttendance(wdf,"wdfStatus");
SSSSS.     checkAttendance(cpp,"cppStatus");
TTTTT.     checkAttendance(dsa,"dsaStatus");
UUUUU. }
VVVVV.

```

```

WWWWW. if(document.getElementById("wdfgrade"))
XXXXX. {
YYYYY.     calculateGrade(wdf,"wdfgrade");
ZZZZZ.     calculateGrade(cpp,"cppgrade");
AAAAAA.     calculateGrade(dsa,"dsagrade");
BBBBBB.     }
CCCCC.
DDDDDD.     function openModal() {
EEEEEE.
FFFFFF.     document.getElementById("courseModal").style.display =
        "block";
GGGGGG.
HHHHHH.     }
IIIIII.
JJJJJJ.     function closeModal() {
KKKKKK.
LLLLLL.     document.getElementById("courseModal").style.display =
        "none";
MMMMMM.
NNNNNN.     }
OOOOOO.

PPPPPP.     // Light / Dark Theme Switcher
QQQQQQ.
RRRRRR.     let themeToggle = document.getElementById("themeToggle");
SSSSSS.
TTTTTT.     // Apply saved theme on page load
UUUUUU.     if (localStorage.getItem("theme") === "dark")
VVVVVV.     {
WWWWWW.         document.body.classList.add("dark-mode");
XXXXXX.         if (themeToggle) themeToggle.innerHTML = "☀";
YYYYYY.     }
ZZZZZZ.
AAAAAAA.     if (themeToggle)
BBBBBBB.     {
CCCCCCC.         themeToggle.addEventListener("click", function()
DDDDDDD.         {
EEEEEEE.             document.body.classList.toggle("dark-mode");
FFFFFFF.
GGGGGGG.             if (document.body.classList.contains("dark-mode"))
HHHHHHH.             {
IIIIIII.                 localStorage.setItem("theme", "dark");
JJJJJJJ.                 themeToggle.innerHTML = "☀";
KKKKKKK.             }

```

```

LLLLLLL.      else
MMMMMMM.      {
NNNNNNN.      localStorage.setItem("theme", "light");
OOOOOOO.      themeToggle.innerHTML = "☺";
PPPPPPP.      }
QQQQQQQ.    });
RRRRRRR.    }
SSSSSSS.
TTTTTTT.    // Hamburger Menu
UUUUUUU.
VVVVVVV.    let hamburger = document.getElementById("hamburger");
WWWWWWW.    let navMenu = document.getElementById("navMenu");
XXXXXXX.
YYYYYYY.    if (hamburger && navMenu)
ZZZZZZZ.    {
AAAAAAA.      hamburger.addEventListener("click", function()
BBBBBBB.      {
CCCCCCC.        navMenu.classList.toggle("active");
DDDDDDD.      });
EEEEEEE.    }
FFFFFFF.

GGGGGGGG.    // Notification Banner
HHHHHHHH.
IIIIIIII.    let notificationBanner =
document.getElementById("notificationBanner");
JJJJJJJJ.    let closeBanner = document.getElementById("closeBanner");
KKKKKKKK.
LLLLLLLL.    if (notificationBanner && closeBanner)
MMMMMMM.    {
NNNNNNNN.      // Keep the banner closed for the rest of the session once
dismissed
OOOOOOOO.      if (sessionStorage.getItem("bannerClosed") === "true")
PPPPPPP.      {
QQQQQQQQ.        notificationBanner.style.display = "none";
RRRRRRRR.      }
SSSSSSSS.
TTTTTTTT.    closeBanner.addEventListener("click", function()
UUUUUUUU.    {
VVVVVVVV.      notificationBanner.style.display = "none";
WWWWWWW.      sessionStorage.setItem("bannerClosed", "true");
XXXXXXX.    });
YYYYYYY.    }

```

```

ZZZZZZZZ.

AAAAAAA. // Collapsible FAQ
BBBBBBBB.
CCCCCCCC. let faqQuestions = document.querySelectorAll(".faq-question");
DDDDDDDD.
EEEEEEEE. faqQuestions.forEach(function(btn)
FFFFFFF. {
GGGGGGGG. btn.addEventListener("click", function()
HHHHHHHH. {
IIIIIIIII. let answer = btn.nextElementSibling;
JJJJJJJJ. answer.classList.toggle("show");
KKKKKKKK.
LLLLLLLLL. let icon = btn.querySelector(".faq-icon");
MMMMMMMM. if (icon)
NNNNNNNN. {
OOOOOOOO. icon.innerHTML = answer.classList.contains("show")
? "-" : "+";
PPPPPPPP. }
QQQQQQQQ. });
RRRRRRRR. });
SSSSSSSS.
TTTTTTTT. // Content / Image Slider
UUUUUUUU.
VVVVVVVV. let slides = document.querySelectorAll(".slide");
WWWWWWWW. let nextSlideBtn = document.getElementById("nextSlide");
XXXXXXXX. let prevSlideBtn = document.getElementById("prevSlide");
YYYYYYYY. let currentSlide = 0;
ZZZZZZZZ. let slideTimer;

AAAAAAA.
BBBBBBBB. function showSlide(index)
CCCCCCCC. {
DDDDDDDD. slides.forEach(function(slide)
EEEEEEEE. {
FFFFFFF. slide.classList.remove("active");
GGGGGGGG. });
HHHHHHHH. slides[index].classList.add("active");
IIIIIIIII. }
JJJJJJJJ.
KKKKKKKK. function startSlider()
LLLLLLLLL. {
MMMMMMMM. slideTimer = setInterval(function()
NNNNNNNN. {
OOOOOOOO. currentSlide = (currentSlide + 1) % slides.length;

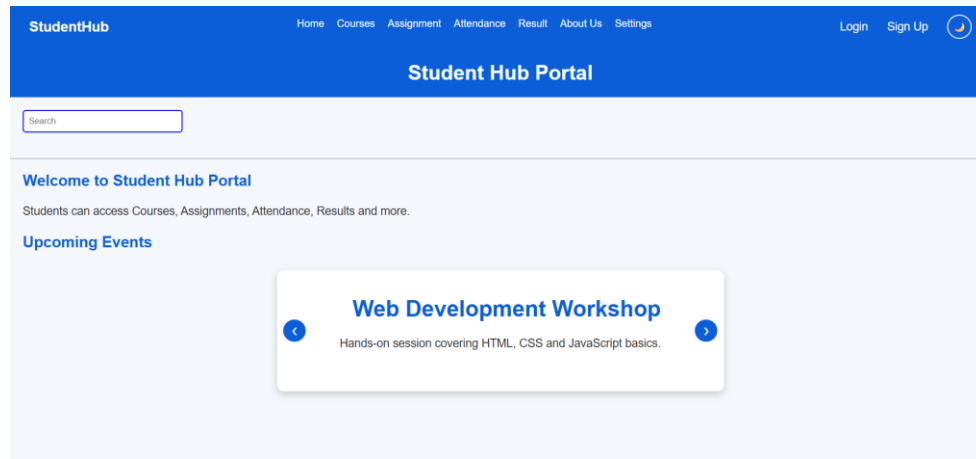
```

```

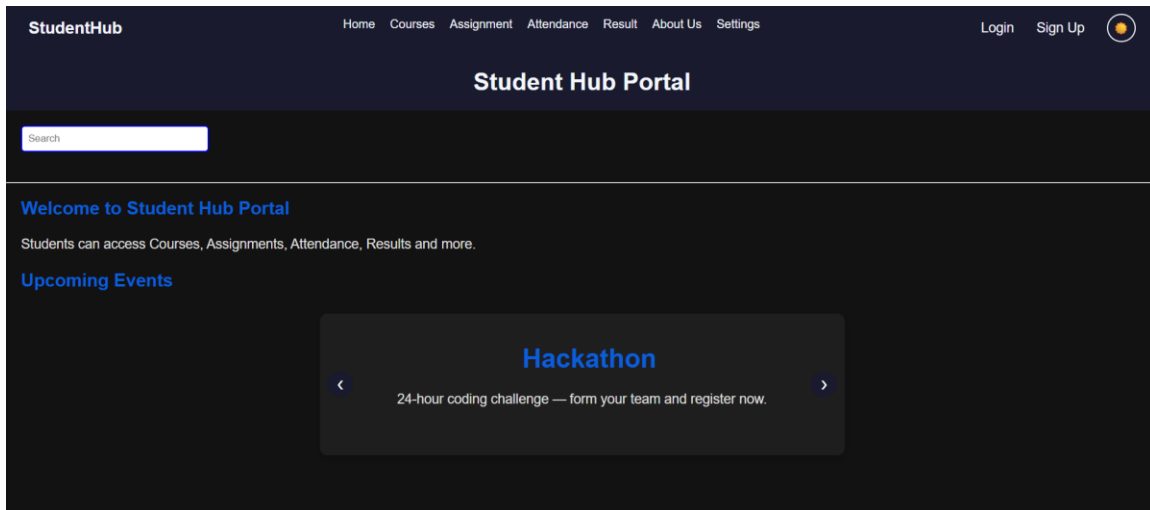
PPPPPPPPPP.      showSlide(currentSlide);
QQQQQQQQQQ.      }, 5000);
RRRRRRRRRR.  }
SSSSSSSSSS.
TTTTTTTTTT.  if (nextSlideBtn && prevSlideBtn && slides.length > 0)
UUUUUUUUUU.  {
VVVVVVVVVV.      nextSlideBtn.addEventListener("click", function()
WWWWWWWWWW.      {
XXXXXXXXXX.          currentSlide = (currentSlide + 1) % slides.length;
YYYYYYYYYY.          showSlide(currentSlide);
ZZZZZZZZZZ.          clearInterval(slideTimer);
AAAAAAAAAAAA.          startSlider();
BBBBBBBBBBBB.      });
CCCCCCCCCCCC.
DDDDDDDDDD.      prevSlideBtn.addEventListener("click", function()
EEEEEEEEEEEE.      {
FFFFFFFFFFFF.          currentSlide = (currentSlide - 1 + slides.length) %
slides.length;
GGGGGGGGGG.          showSlide(currentSlide);
HHHHHHHHHH.          clearInterval(slideTimer);
IIIIIIIIIII.          startSlider();
JJJJJJJJJJ.      });
KKKKKKKKKK.
LLLLLLLLLLLL.      startSlider();
MMMMMMMMMM.  }
NNNNNNNNNN.

```

C. Evidence of Work (Screenshots)



Description: This figure shows the StudentHub Portal home page in light mode. A blue navigation bar at the top contains the StudentHub logo, navigation links, Login and Sign Up options, and a theme toggle. Below it is a search bar, a welcome message, and an Upcoming Events section. The event carousel displays a Web Development Workshop with a short description and left/right navigation buttons.



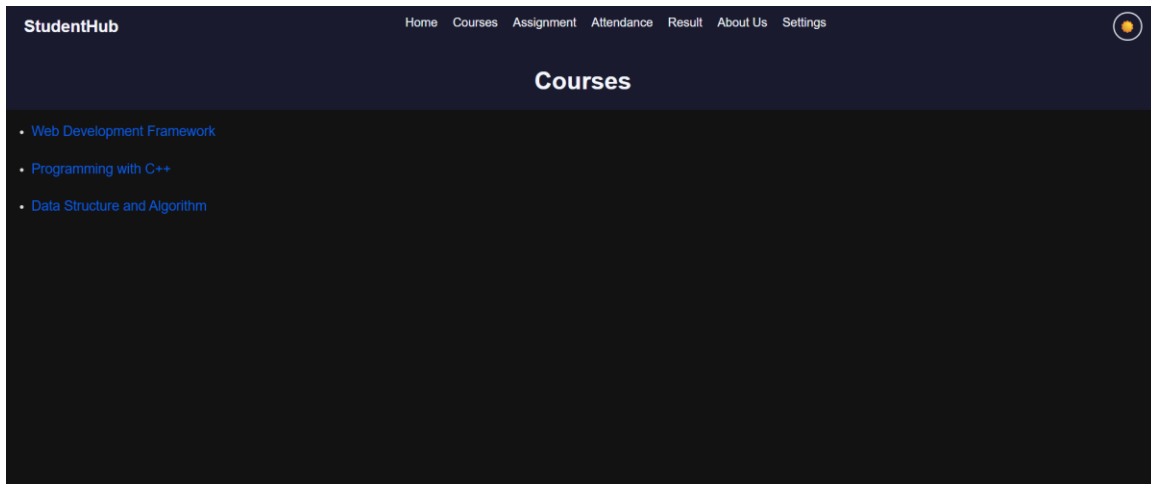
Description: This figure shows the StudentHub Portal home page in dark mode. The interface uses a dark background with a dark navy header while blue is used for headings and highlights. The page contains the same navigation menu, search bar, welcome section, and upcoming events carousel. This time, the featured event is a Hackathon, described as a 24-hour coding challenge.

The screenshot shows the StudentHub Login Page in dark mode. The top navigation bar is dark blue with the 'StudentHub' logo on the left, a 'Home' link in the center, and a theme toggle icon on the right. Below the navigation bar is a dark blue banner with the text 'Welcome To StudentHub'. The main content area is dark gray and features a centered login form. The form has a title 'Login Page' in blue. It contains a 'Username:' label, a text input field with the value '25dce107', a 'Password:' label, a text input field with the placeholder 'Enter the Password', a blue 'Login' button, a validation message 'Please enter the Password' in red, and a link 'Don't have an account? Sign Up' in blue.

Description: The figure shows the StudentHub Login Page in dark mode. The page has a simple top navigation with the StudentHub logo, a Home link, and a theme toggle. In the center is a login form containing a Username field, Password field, and Login button. A validation message appears below the form, along with a Sign Up link for users who do not have an account.

The screenshot shows the StudentHub Sign Up/Registration Page in dark mode. The top navigation bar is dark blue with the 'StudentHub' logo on the left, a 'Home' link in the center, and a theme toggle icon on the right. Below the navigation bar is a dark blue banner with the text 'Welcome to StudentHub'. The main content area is dark gray and features a centered sign up form. The form has a title 'Sign Up' in blue. It contains an 'Email:' label, a text input field with the value '25dce107@charusat.edu.in', a 'Create Password:' label, a text input field with the placeholder 'Create the Password', a 'Confirm Password:' label, a text input field with the placeholder 'Enter the password', a blue 'Register' button, and a validation message 'Please enter the Password' in red.

Description: The figure shows the StudentHub Sign Up/Registration Page in dark mode. It has a centered registration form with fields for Email, Create Password, and Confirm Password. A Register button is provided to submit the form, and validation text is displayed below it. The page also includes the StudentHub logo, a Home link, and a theme toggle in the top navigation.

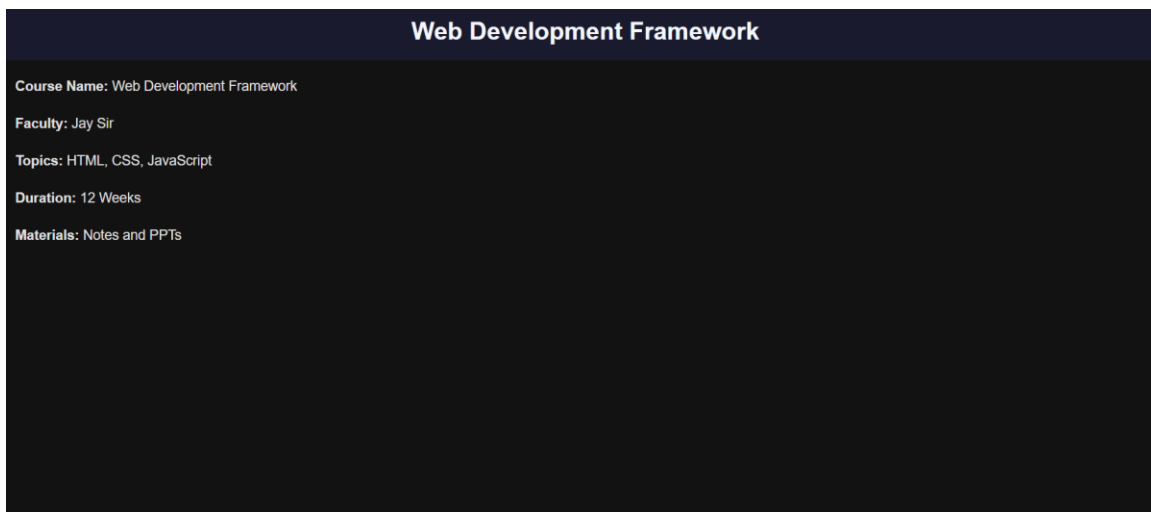


Description: The figure shows the list of courses available to the student. The page uses a dark-themed interface with a navigation bar containing links to Home, Courses, Assignment, Attendance, Result, About Us, and Settings.

Three courses are displayed:

- Web Development Framework
- Programming with C++
- Data Structure and Algorithm

The course names are presented as clickable links, allowing the student to select a particular course and view its details.

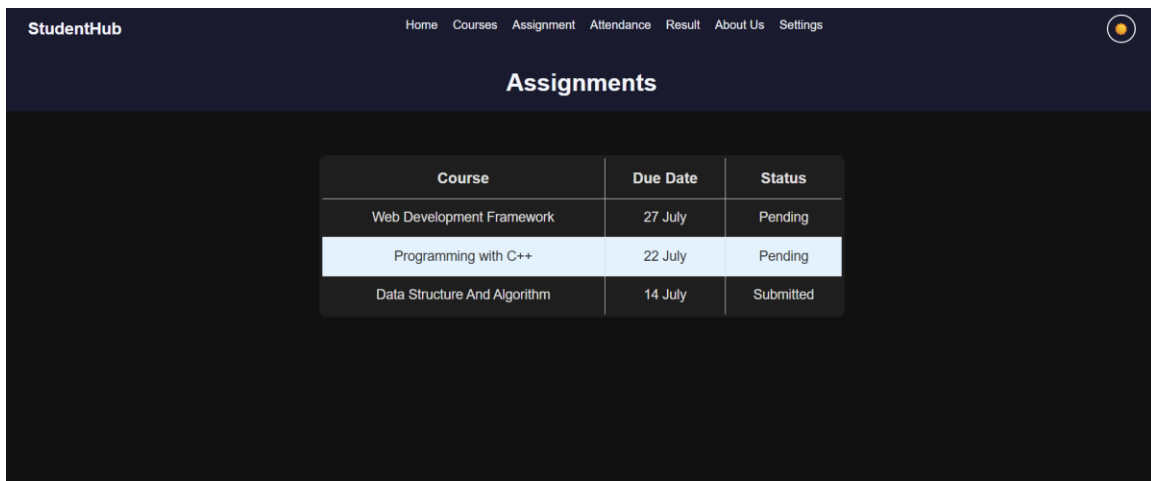


Description: The figure shows the detailed information about the selected course.

It contains:

- **Course Name:** Web Development Framework
- **Faculty:** Jay Sir
- **Topics:** HTML, CSS, JavaScript
- **Duration:** 12 Weeks
- **Materials:** Notes and PPTs

The page demonstrates how JavaScript/HTML navigation can be used to move from a course list to a specific course-information page.

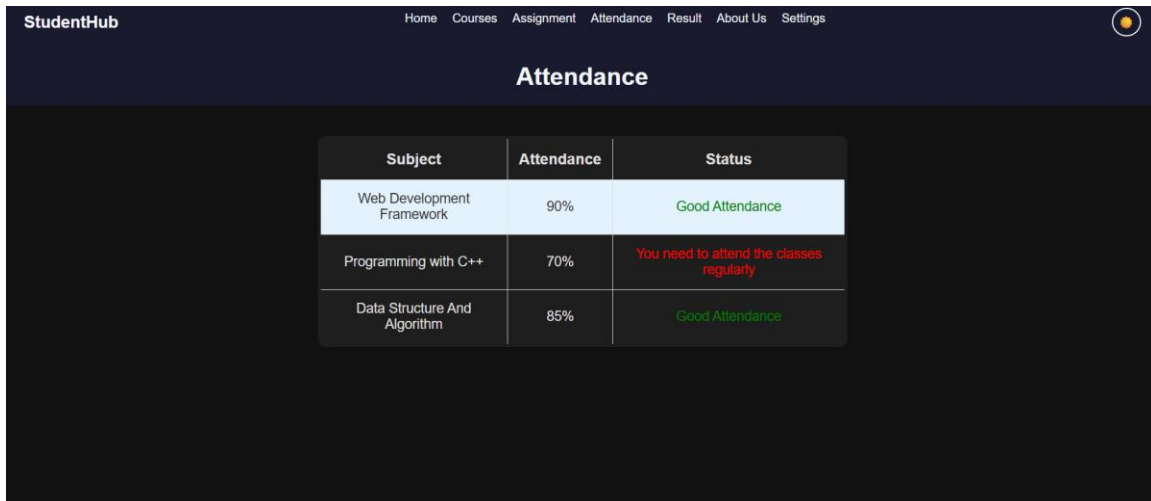


Course	Due Date	Status
Web Development Framework	27 July	Pending
Programming with C++	22 July	Pending
Data Structure And Algorithm	14 July	Submitted

Description: This figure shows the assignment information in a structured table. The table contains three columns:

- Course
- Due Date
- Status

The table makes it easy for students to identify upcoming deadlines and check whether an assignment has been submitted.



Attendance		
Subject	Attendance	Status
Web Development Framework	90%	Good Attendance
Programming with C++	70%	You need to attend the classes regularly
Data Structure And Algorithm	85%	Good Attendance

Description: This figure shows the student's attendance for different subjects.

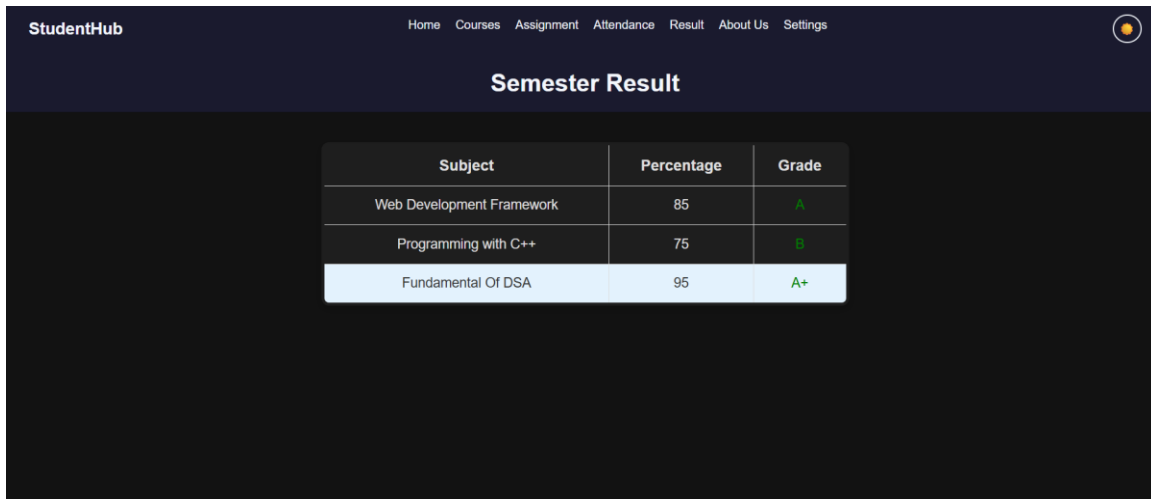
It contains three columns:

- Subject
- Attendance
- Status

The displayed information includes:

- Web Development Framework – **90%** – **Good Attendance**
- Programming with C++ – **70%** – **You need to attend the classes regularly**
- Data Structure and Algorithm – **85%** – **Good Attendance**

The page uses different status messages to provide immediate feedback about the student's attendance.



Subject	Percentage	Grade
Web Development Framework	85	A
Programming with C++	75	B
Fundamental Of DSA	95	A+

Description: This figure shows the the student's academic performance in a table.

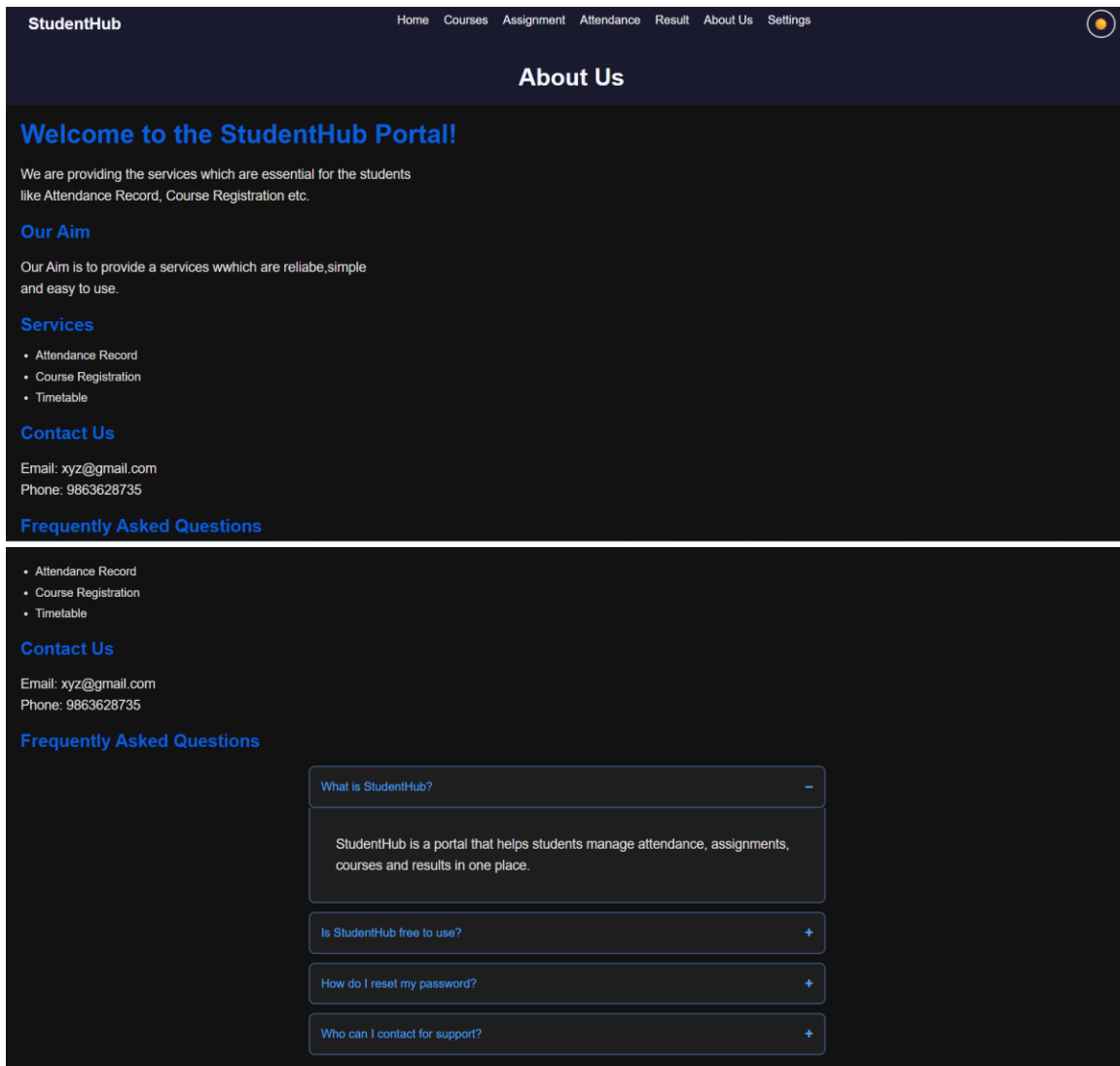
The table includes:

- Subject
- Percentage
- Grade

The results shown are:

- Web Development Framework – 85% – A
- Programming with C++ – 75% – B
- Fundamental of DSA – 95% – A+

The grades are visually highlighted, making it easier for the student to understand their performance.



Description: This figure shows the information about the StudentHub portal and its purpose.

It contains sections such as:

Welcome to the StudentHub Portal

The page explains that StudentHub provides services essential for students, including attendance records and course registration.

Our Aim

The aim is to provide services that are **reliable, simple, and easy to use**.

Services

The portal provides services such as:

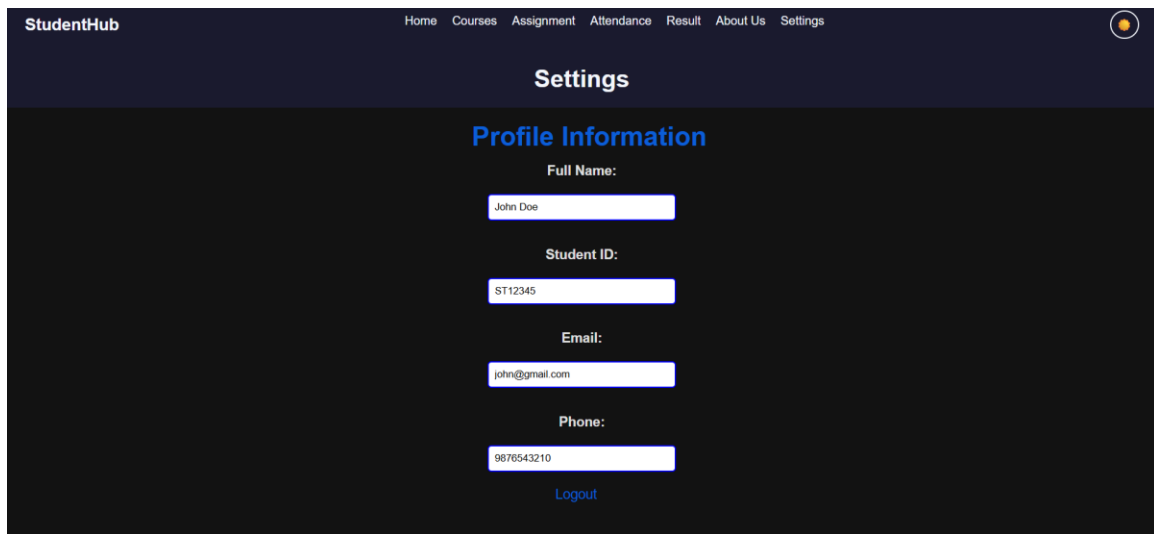
- Attendance Record
- Course Registration
- Timetable

Contact Us

The page provides contact information, including an email address and phone number.

Frequently Asked Questions

The page also contains an interactive FAQ section where questions can be expanded or collapsed.



The screenshot displays the 'StudentHub' interface. At the top, a navigation bar includes links for Home, Courses, Assignment, Attendance, Result, About Us, and Settings. The 'Settings' page is active, showing a 'Profile Information' section. This section contains four input fields: 'Full Name' (John Doe), 'Student ID' (ST12345), 'Email' (john@gmail.com), and 'Phone' (9876543210). A 'Logout' button is located at the bottom of the form.

Description: This figure shows the profile information for the student.

It contains fields for:

- Full Name
- Student ID
- Email
- Phone

The page also provides a Logout option.

The profile information is presented in a centered form-like layout, making it easy for users to view or manage their information.

D. GitHub Repository Link:

<https://github.com/devanshirshah24-pixel/WDF-Practicals/tree/main/Practical4>

E. Challenges Faced:

1. Understanding DOM Manipulation

One of the initial challenges was understanding how JavaScript interacts with HTML elements through the DOM. Selecting the correct elements and changing their content, classes, or visibility required careful implementation.

2. Implementing Event Listeners

Attaching click events to buttons, navigation elements, FAQ sections, and the theme switcher was challenging. I had to ensure that the correct JavaScript function was executed whenever the user interacted with an element.

3. Creating the FAQ Accordion

Implementing the expandable and collapsible FAQ section required manipulating the visibility of the answer elements. I also had to make sure that the + and – indicators changed correctly.

4. Implementing Light/Dark Mode

Creating the theme-switching functionality was another challenge. The CSS had to support both themes, while JavaScript needed to add or remove the appropriate class from the page.

5. Maintaining Theme Preference

Using `localStorage` to remember the user's selected theme required understanding how data can be stored in the browser and retrieved when the page is loaded again.

6. Maintaining Consistent Design

Keeping the same navigation bar, fonts, spacing, buttons, tables, and colors across multiple pages was challenging. Changes to CSS had to be tested across different pages to avoid affecting the layout unexpectedly.

7. Handling Page Navigation

Connecting the different pages, such as Courses, Assignments, Attendance, Results, About Us, and Settings, required ensuring that each navigation link correctly opened the intended page.

8. Responsive and Usable Interface

Making sure that tables, forms, FAQ sections, and other components remain readable and usable on different screen sizes required additional testing and CSS adjustments.

9. Debugging JavaScript Errors

During development, identifying errors in JavaScript event handling and DOM selection was challenging. Browser developer tools and the console were useful for finding and correcting these problems.

F. Learning Outcomes:

- Understand DOM manipulation and use JavaScript to select and modify HTML elements dynamically.
- Implement event handling using JavaScript event listeners for buttons, links, FAQ sections, and other interactive components.
- Create interactive UI components, such as collapsible FAQs, sliders, theme switches, and dynamic content.
- Use CSS classes with JavaScript to dynamically change the appearance and behavior of web elements.
- Implement light/dark mode functionality to improve the user experience.
- Understand and use localStorage for storing user interface preferences such as the selected theme.
- Develop a multi-page student portal containing courses, assignments, attendance, results, profile settings, and information pages.
- Improve frontend usability by providing clear navigation, visual feedback, and interactive components.
- Debug JavaScript applications using browser developer tools and the browser console.

- Understand the relationship between HTML, CSS, and JavaScript in creating dynamic and interactive web applications.

Faculty Remarks: (Keep this blank for faculty)